Announcements

- Team Formation CATME Survey Due Date 8/30. Form teams on Canvas groups if you want to.
- AI survey extra credit due 8/30. Please fill the survey out.
- Office Hours will begin on 8/31 – in CCB 267
- All Project 1 related assignments have released
  - Planning Assignment
  - Progress Report
  - Requirements
  - Presentation, Demo, Code, Report
  - Design
- Project Teams and Mentors will be assigned by 31st.
- GCP Credits are Out. Please avail the coupon.

CS 3300 A Introduction to Software Engineering

Lecture 02: Project Description; Prompt Engineering

Dr. Nimisha Roy ▸ nroy9@gatech.edu

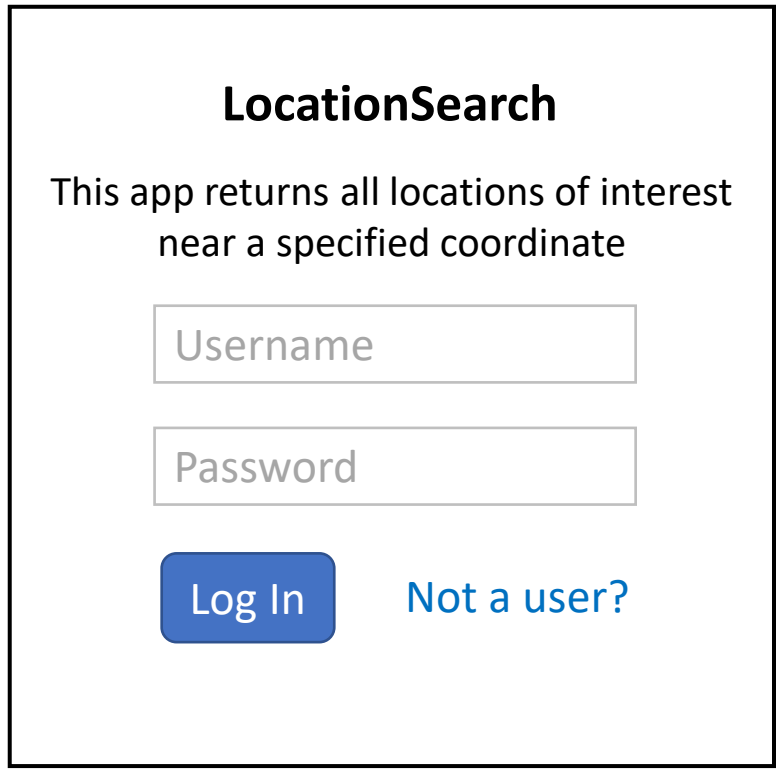
Project Description

LocalLens - *Discover what's around you*

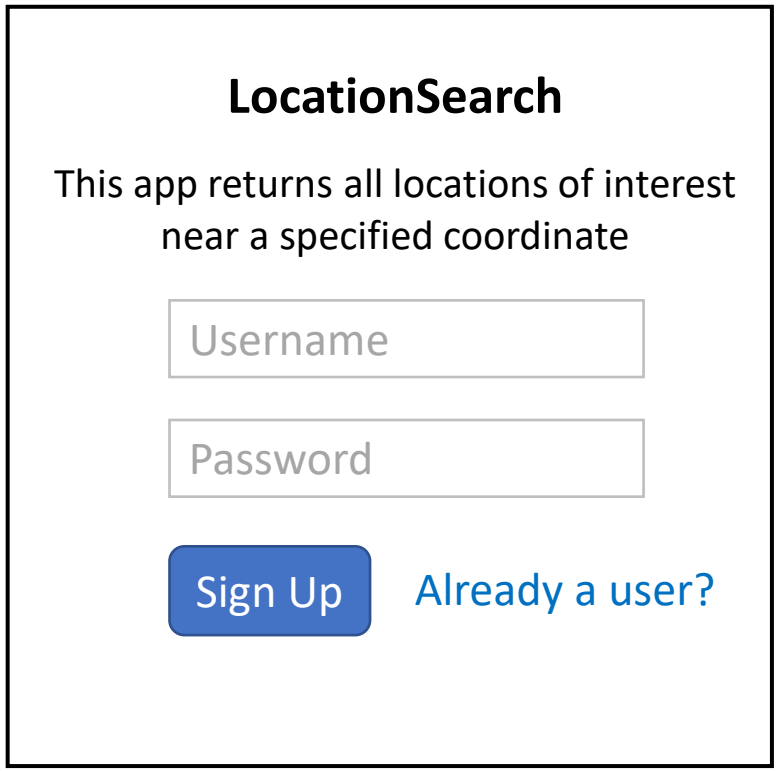
Problem Statement

Build a web application that helps users discover interesting places near a specified location.
Given a location and search radius, users should be able to search, explore, and filter nearby places.
You decide how to make the experience useful.

LANDING PAGE



LANDING PAGE



What Must LocalLens Do? (Required Functionality)

Every team must implement these core capabilities:

- 🔍 **Search**
  Enter a location or coordinates and a search radius to find nearby places.

- 📍 **Explore Results**
  Present useful information about the places returned using an interactive **map, and some text (e.g. list)**.

- 🎯 **Filter**
  Allow users to filter results by at least **one type of place**
  Restaurants • *Parks* • *Attractions* • *Hotels* • *Hospitals* • *etc.*

That's the required product functionality. You should also have a landing page with user authentication.

You need to implement atleast 2 more features of your choice.

How you design the experience — and what else you build — is up to your team.

Example UI

USER PAGE

Select a Search Area

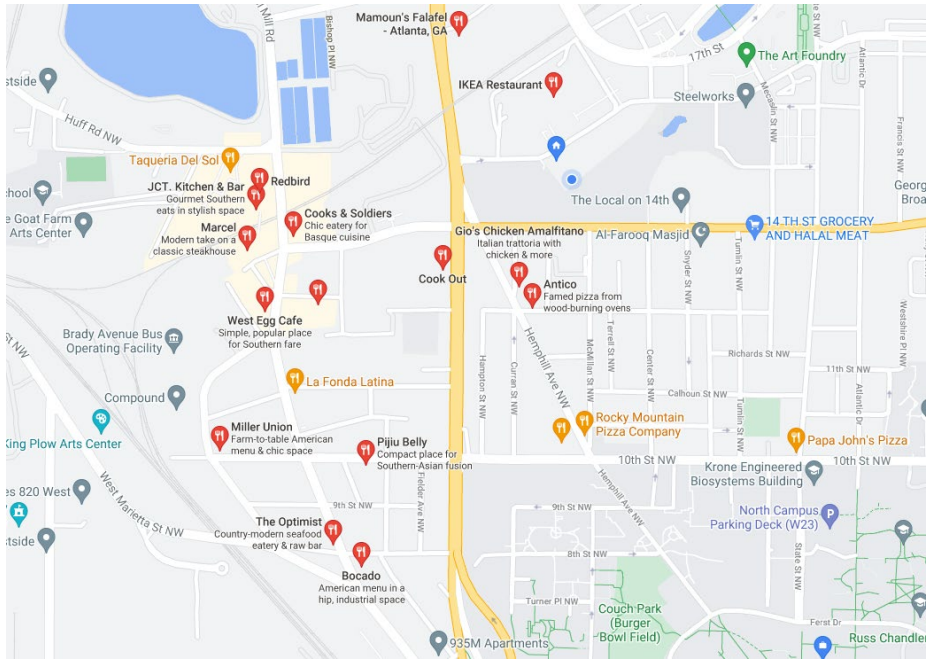
Latitude: Longitude: Search radius:

Find Places!

Search Results

Show 10 entries Search

Name	Distance to Point (ft)	Address	Rating	Price Level
JCT Kitchen & Bar	1815	1198 Howell Mill Rd #18, Atlanta, GA 30318	4/5	$$
Piju Belly	…	…	…	…
Miller Union	…	…	…	…
West Egg Cafe	…	…	…	…



Log Out

Mandatory Features

Landing Page + Authentication

- Landing page introducing LocalLens
- User sign-up / login

Search

- Location/coordinates + search radius
- Input validation

Explore Results

- Interactive map with returned locations
- Text-based display of useful location information

Filter

- At least **one location-type filter**
- *Restaurants • Parks • Attractions • Hotels • Hospitals • etc.*

Examples of additional features

- **Personalized Recommendations**
  Recommend places based on user preferences, previous activity, ratings, or other relevant information.
- **Favorites & User Collections**
  Allow authenticated users to save, organize, view, and remove favorite places.
- **Search History & Revisit**
  Persist previous searches and allow users to revisit/re-run them.
- **Trip / Outing Planner**
  Select multiple locations and create an itinerary or route.
- **Reviews & Ratings**
  Allow users to rate/review locations and view contributions from other users.
- **Smart / Natural-Language Search**
  Support queries such as *“Find highly rated inexpensive restaurants within walking distance.”*
- **Team-Proposed Feature**
  Propose another feature of **comparable scope and complexity** for approval to your TA mentor.

Technology Suggestions

GOOGLE CLOUD PLATFORM*

Offers services for compute, storage, networking, big data, machine learning and the internet of things (IoT), as well as cloud management, security and developer tools.

BACKEND SERVER ARCHITECTURE

- SpringBoot* – Framework for developing the backend server architecture and to run the server application easily
- Maven* - Used for dependency injection and management as a Java package manager. Provided as a part of central repository in Spring
- Mockito – Test stub method for unit testing SpringBoot application
- VS Code/IntelliJ - IDE

DATABASE

- GCP DataStore
- MySQL
- Google Storage

*** Must use**

Technology Suggestions

API

- Google Places API - To implement an interactive, searchable map
- Google Distance Matrix API/ OpenStreetMap API

FRONTEND

- Thymeleaf with Springboot - Java template engine for both web and standalone environments. Set of Spring integrations.
- Leaflet/jQuery/Bootstrap -- Open-source JavaScript library for creating interactive maps/implementing HTML document traversal and manipulation, event handling, animation, and Ajax/CSS framework to create modern websites and web apps
- React.JS - open-source JavaScript library used for building user interfaces for single-page applications

VERSION CONTROL

GitHub*

AI Tools for Implementation

GPT API*

Copilot, Claude, codex

*** Must use**

Some Considerations/Functionalities

Password Security

- Salting & Hashing Passwords
- JWT – Validity of Tokens (optional)

Database

- Google DataStore is recommended.
- Google storage has some problem: 2 people simultaneous logging in may cause overwrite

Webjars

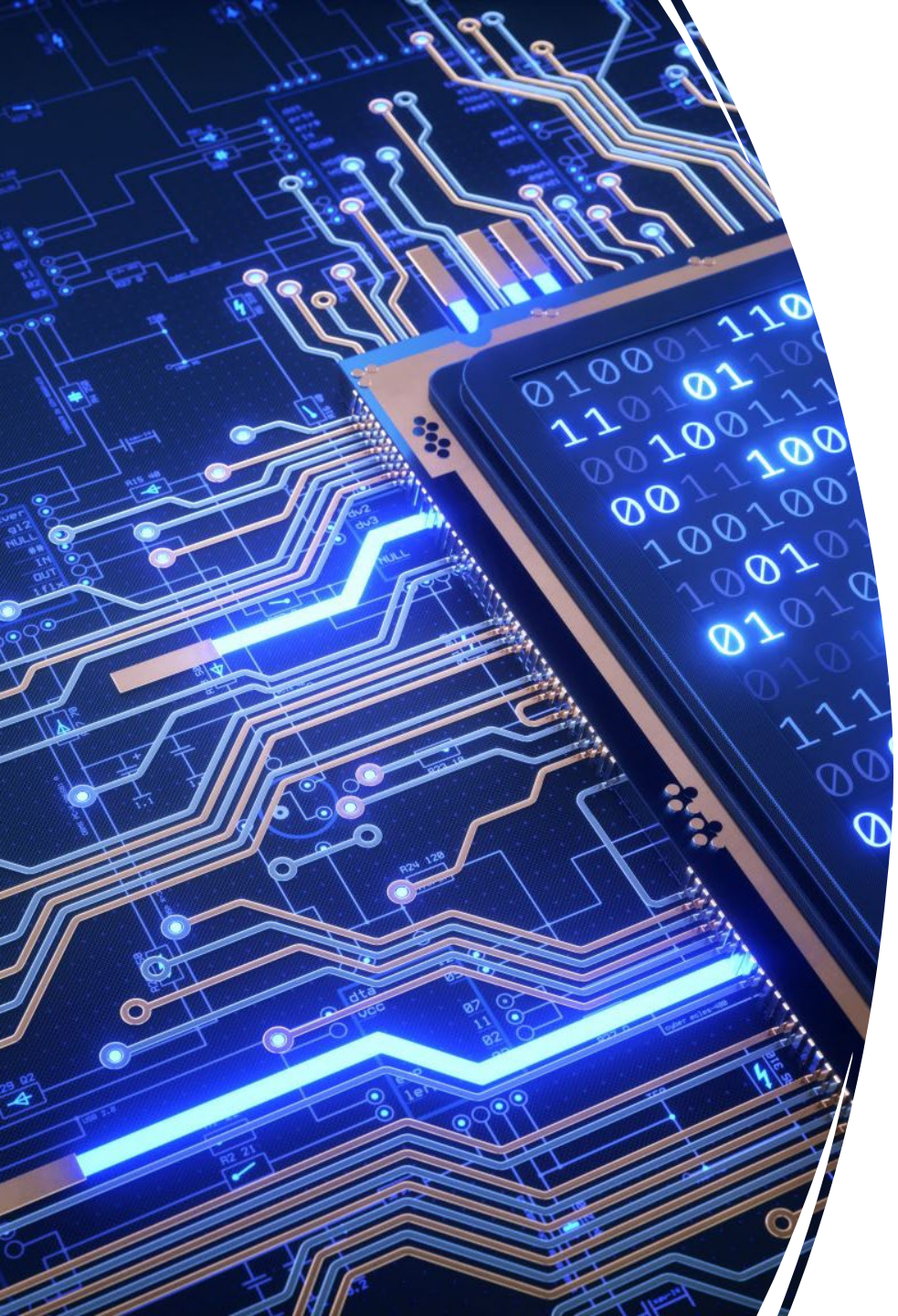
Use webjars - client-side web libraries packaged into JAR (Java Archive) files to easily manage web dependencies

Testing

Important. Modules Included in Google App Engine.

Project 1
Planning and Progress Report

Prompt Engineering



Introduction to Prompt Engineering in Software Engineering

•**Prompts** involve instructions and context passed to a language model to achieve a desired task.

•**Prompt engineering** is the practice of developing and optimizing prompts to efficiently use large language models (LLMs) for a variety of applications.

- For software engineers, a prompt can act like a lightweight specification:
- **What should happen? What context matters? What constraints must be respected? What output do you need?**

Why does Prompting Still Matter?

Modern AI can produce impressive results from very simple prompts.

So the problem is **not simply: Bad prompt → Bad code**

- A vague prompt may produce code that works.
- The bigger issue is that **AI fills in what you don't specify.**
- That means it may make assumptions about:
  - Requirements
  - Architecture
  - APIs and interfaces
  - Dependencies
  - Error handling
  - Scope of the change

More ambiguity → more engineering decisions delegated to AI

The Problem with “Just Make it Work”

AI Often Optimizes for the Immediate Task

Ask: “Fix this bug.” AI may produce a fix that:

✓ Resolves the immediate failure
✓ Compiles
✓ Passes the visible test

But it may also:

- Duplicate existing logic
- Violate architectural boundaries
- Introduce unnecessary dependencies
- Create inconsistent patterns
- Ignore maintainability
- Increase coupling
- Create future technical debt

A locally correct solution can still be a poor system-level solution.

Vague vs. Structured Prompt

Vague Prompt

Implement favorites for LocalLens.

Structured Prompt

Implement a favorites feature in our SpringBoot LocalLens application. Favorites belong to authenticated users and must persist between sessions. Follow the existing Controller → Service → Repository architecture. Do not change the existing Place API or authentication interfaces. Include tests for adding, removing, and retrieving favorites.”

Structured Prompt gives the engineer more control over:
Scope | Architecture | Constraints | Expected behavior

Prompt Engineering Techniques

Explicit Constraints: Define what the AI must preserve, follow, or avoid to control the scope of its solution.

Zero/Few shot Prompting: Provide zero, one, or multiple examples to clarify expected behavior and reduce ambiguity.

Prompt Chaining: Break a complex task into smaller connected steps, using the output of one interaction to guide the next.

Iterative Refinement: Review, critique, and refine AI output based on requirements, constraints, tests, and engineering judgment.

Technique 1: Explicit Constraints

Example:

"Refactor searchPlaces():
Do not change its public interface
Do not modify the controller
Do not introduce new dependencies
Preserve existing behavior
Keep all current tests passing"

- **Constraints are engineering guardrails.**
- They help prevent AI from expanding the scope of a seemingly small task.

Technique 2: Zero/One/Few Shot Prompting

Show AI the behavior you expect

- **Zero-shot**
  Instruction with no examples
- **One-shot**
  Provide one example
- **Few-shot**
  Provide multiple examples

For LocalLens:

Input: radius = 5 → Search normally
Input: radius = -1 → Validation error
Input: invalid latitude → Validation error

- Examples can function like **lightweight test cases inside the prompt.**

Few Shot Prompting: Another Example

- **Query**: Generate a function that can calculate and return a response status for an API call.
- **Example 1:** A function getResponseStatus returns "Success" if an API call response status is 200, and "Error" otherwise. An example of this function is

```
def getResponseStatus(response_code):
    return "Success" if response_code == 200 else "Error"
```

- **Example 2**: The "checkApiCall" function is designed to make an API call and use getResponseStatus to determine and print the status.

Few Shot Prompting: Another Example

- **Output**: Here's a Python code snippet that calculates and returns the response status for an API call:

```python
import requests

def getResponseStatus(response):
    if response.status_code == 200:
        return "Success"
    elif response.status_code == 404:
        return "Not Found"
    else:
        return "Error"

def checkApiCall(url):
    try:
        response = requests.get(url)
        status = getResponseStatus(response)
        return f"API Call Status: {status}"
    except requests.exceptions.RequestException as e:
        return f"Failed to make API call: {str(e)}"

# Example usage
url = "https://api.example.com/data"
result = checkApiCall(url)
print(result)
```

Technique 3: Prompt Chaining

- Prompt Chaining is a technique where **the output of one prompt serves as the input or context for the next**, creating a sequence of interlinked prompts that guide the AI towards a complex goal
- Break complex tasks into smaller interactions - Enables the handling of multi-step problems within AI systems by maintaining a contextual flow, improving accuracy in tasks requiring deep contextual understanding

- **Example:**
- Instead of: “Build the LocalLens favorites feature.”

Prompt Chaining

Use:

1. Analyze

How should favorites fit into our existing architecture?

↓

2. Design

Propose the necessary classes/interfaces.

↓

3. Implement

Implement the approved design.

↓

4. Test

Generate tests for expected and edge-case behavior.

↓

5. Review / Refine

Prompt Chaining Example

Initial Prompt:

- **Query:** "Generate a Python function to parse and summarize JSON data from a project management tool's API."
- **Chain Prompt 1:**

Output from Initial Prompt:

```
def parse_json(json_data):
    """Extracts project details from JSON data."""
    projects = json.loads(json_data)
    summary = {project["id"]: project["name"] for project in projects}
    return summary
```

Prompt Chaining Example

- **Follow-up Query:** "How would you use this function to display a formatted list of project names and their statuses?"
- **Chain Prompt 2:**
- **Output from Chain Prompt 1:**

```
def display_projects(json_data):
    projects = parse_json(json_data)
    for project_id, project_name in projects.items():
        print(f"Project ID: {project_id}, Project Name: {project_name}")


# Example JSON Data
json_data = '''
[
  {"id": 1, "name": "Project Alpha", "status": "Active"},
  {"id": 2, "name": "Project Beta", "status": "Completed"}
]
'''


display_projects(json_data)
```

Prompt Chaining Example

- **Follow-up Query:** "Now, create a function that updates the status of a specific project by project ID in the project list."
- **Chain Prompt 3:**
- **Output from Chain Prompt 2:**

```
def update_project_status(projects_json, project_id, new_status):
    projects = json.loads(projects_json)
    for project in projects:
        if project['id'] == project_id:
            project['status'] = new_status
            break
    return json.dumps(projects)


# Updated usage
updated_json = update_project_status(json_data, 1, 'Completed')
print(updated_json)
```

Technique 4: Iterative Refinement

The first output is a draft—not the end of the interaction.

Review the output and refine based on what you find:

- “This changes an existing interface. Preserve it.”
- “Handle null input.”
- “You duplicated logic already in PlaceService.”
- “Preserve the existing architecture.”
- “Add tests for these edge cases.”
- “Remove the unnecessary abstraction.”

Generate → Review → Refine → Test

Iteration should improve the solution - not become a sequence of “fix this” prompts.

A Practical Prompting Template

TASK

What do you want AI to do?

CONTEXT

What does AI need to know about the system?

CONSTRAINTS

What must it preserve or avoid?

OUTPUT

What should it produce?

TESTS / SCENARIOS

How should the expected behavior be validated?

Putting it All Together

Scenario: You are adding a feature to an existing library management system.

TASK: Implement a method that allows a user to borrow a book.

CONTEXT

- Java 17 application
- Books have an availability status
- Users may borrow up to 5 books at a time
- Existing Book and User classes must be used

CONSTRAINTS

- Do not change existing public method signatures
- Do not introduce new classes
- A book cannot be borrowed if unavailable
- A user cannot exceed the 5-book limit
- Do not duplicate existing validation logic

OUTPUT

•Implement the borrowBook() method using the existing classes.

TESTS / SCENARIOS

Include tests for:

•Successful borrowing
•Unavailable book
•User already has 5 books
•Invalid/null book
•Invalid/null user

Task → Context → Constraints → Output → Tests

How should you experiment with AI in assignments?

In your reflections, think about:

- Does adding system context change the solution AI proposes?
- Do explicit constraints reduce unnecessary code changes?
- Do examples improve generated tests?
- Does prompt chaining produce better results than one large prompt?
- **How often does AI make reasonable - but incorrect - assumptions?**
- **Do repeated spot fixes introduce duplication or technical debt?**
- Does asking AI to analyze before implementing change the solution?

Collect evidence.

Prompt → Output → Evaluate → Refine → Compare

Using Gen-AI Responsibly in Development

AI Can Produce Good Code - But Still Make Wrong Assumptions

- A solution may work while making unintended decisions about architecture, dependencies, interfaces, or scope.
- Immediate fixes may solve the visible problem while introducing **technical debt** elsewhere.

Avoid the "Spot-Fix" Loop

- Don't repeatedly ask AI to *"fix this error"* without understanding the underlying problem.
- Consider how each change fits into the **larger system and existing design**.

Engineer the Output

- Provide relevant context and constraints.
- Review assumptions and design decisions.
- Test generated code and consider regressions.
- Refine based on engineering evidence, not just whether the immediate error disappears.